Realizado por:
Luis Fornaris
Jose Narvaez
Cesar Benitez
Jorge Melendez
Cesar Fawcett

Presentación

TRAZABILIDAD DISTRIBUIDA CON SPRING CLOUD SLEUTH Y OPENTELEMETRY

INTRODUCCIÓN

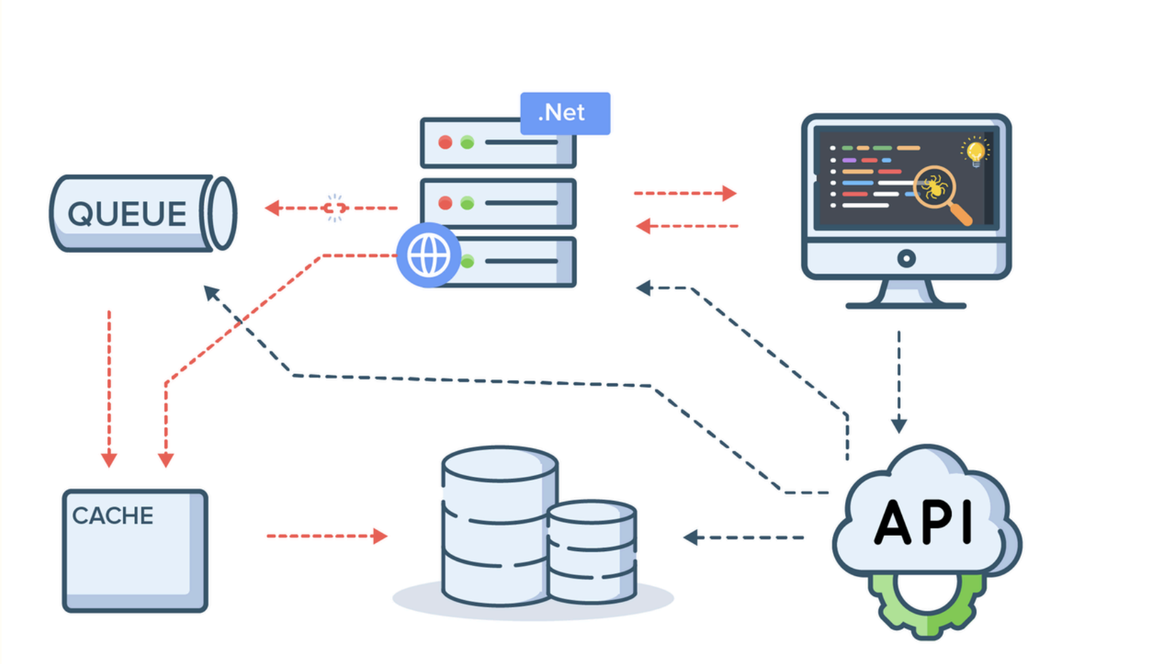
Los microservicios permiten arquitecturas escalables y flexibles, pero rastrear las solicitudes a medida que viajan a través de múltiples servicios puede ser un reto. El rastreo distribuido con OpenTelemetry y Spring Cloud Sleuth nos permite rastrear las solicitudes a través de los servicios para una mejor visibilidad y una depuración más sencilla.

¿Qué es tracing distribuido?

El tracing distribuido es una técnica de observabilidad(Tecnicas que se encargan de la monitorización y el analisis de registros y anomalías) que permite seguir el recorrido de una solicitud (request) a través de múltiples servicios y procesos en arquitecturas distribuidas, típicamente basadas en microservicios.

- ## De que trata?

El tracing distribuido consiste en rastrear de extremo a extremo cómo una petición viaja entre servicios y bases de datos de un sistema distribuido, identificando cada operación (span) y midiendo su duración.



Sleuth en Spring Cloud: autoconfiguración, IDs de trazas y spans

Spring Cloud Sleuth : es una biblioteca que proporciona tracing distribuido para aplicaciones que utilizan Spring Boot. Está diseñada para integrar con sistemas de tracing distribuido como Zipkin y OpenTelemetry. Algunas de las características principales son

TraceId: Cada solicitud que entra en el sistema recibe un ID único de traza.

Spans: Cada servicio que maneja una solicitud crea su propio span.

Autoconfiguración: Sleuth se integra automáticamente con las aplicaciones Spring Boot sin necesidad de configuraciones complicadas.

```
INFO [AuthService, TraceID: abc123, SpanID: 1] User authenticated
INFO [OrderService, TraceID: abc123, SpanID: 2] Order created
INFO [InventoryService, TraceID: abc123, SpanID: 3] Item available
INFO [PaymentService, TraceID: abc123, SpanID: 4] Payment processed
```

OpenTelemetry

OpenTelemetry (OTel) es un estándar CNCF que unifica trazas, métricas y logs, soportando múltiples protocolos (OTLP, Jaeger, Zipkin), es un conjunto de herramientas que permiten recopilar, procesas y exportar datos de observanbilidad como trazas(traces), metricas(metrics) y registros(logs).

Su objetivo es ayudar a entender el comportamiento de las aplicaciones.

Ventajas:

- Soporta trazas, métricas y logs en una sola API.
- Es un estándar abierto, por lo que puede integrarse con muchas soluciones de monitoreo.
- Se está convirtiendo en el estándar de facto en la comunidad de observabilidad.
- Es compatible con entornos y servicios y evita que mezcle multiples herramientas incompatibles.

Supervisión y resolución de problemas con Zipkin

Es una herramienta de trazabilidad distribuida que permite seguir el recorrido de una solicitud a través de múltiples microservicios.

"Te permite ver lo que el sistema realmente hace, no solo lo que crees que hace."

Como se usa

- INTEGRADO CON OPENTELEMETRY O MICROMETER TRACING
- LAS TRAZAS SE ENVÍAN AUTOMÁTICAMENTE DESDE CADA MICROSERVICIO
- ZIPKIN MUESTRA LOS RESULTADOS EN UNA INTERFAZ WEB VISUAL

ZIPKIN

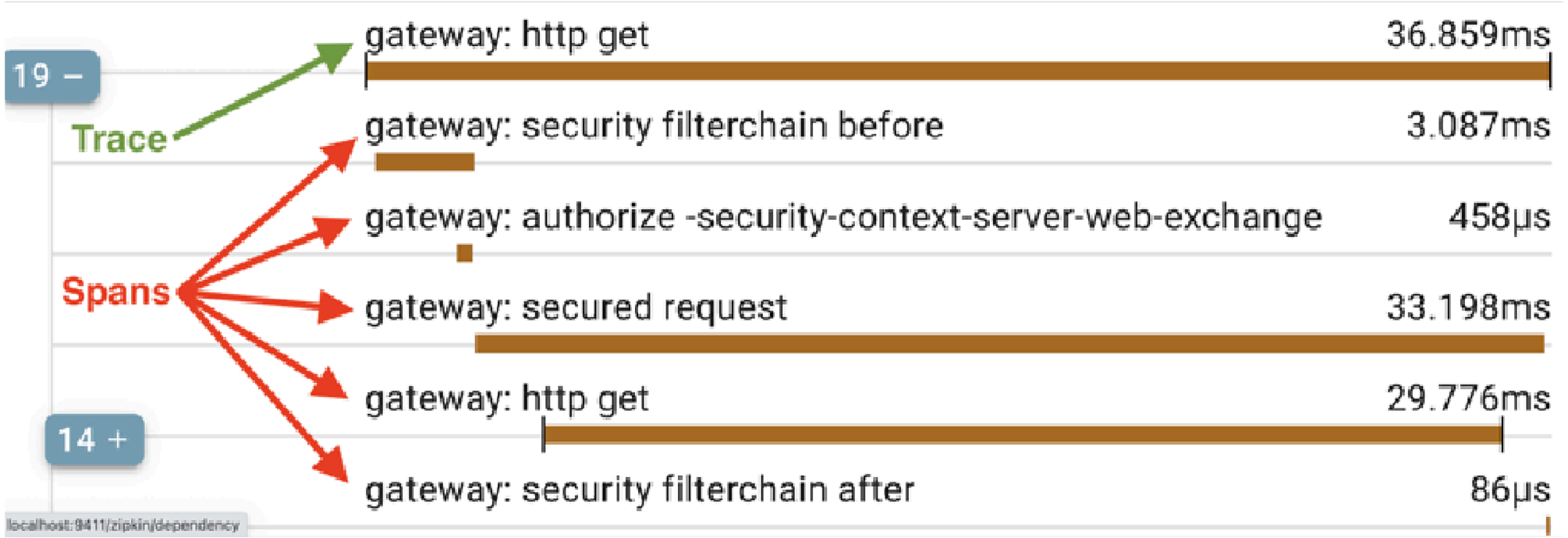


Figure 14.1: Example of a trace with its spans

IMPLEMENTACIÓN

Spring Cloud Sleuth 3.1.9

OVERVIEW LEARN SUPPORT SAMPLES

Spring Cloud Sleuth's last minor version is 3.1. You can check the 3.1.x branch for the latest commits. The core of this project got moved to Micrometer Tracing project and the instrumentations will be moved to Micrometer and all respective projects (no longer all instrumentations will be done in a single repository).

AGREGAR LAS SIGUIENTES DEPENDENCIAS EN LOS SERVICIOS

```xml
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>io.micrometer</groupId>
            <artifactId>micrometer-tracing-bom</artifactId>
            <version>${micrometer-tracing.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>

<dependencies>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-tracing</artifactId>
    </dependency>
</dependencies>
```

OpenTelemetry Tracer

```xml
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
```

OpenZipkin Zipkin with OpenTelemetry

```xml
<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-zipkin</artifactId>
</dependency>
```

docker-compose.yml

```yaml
zipkin:
  image: openzipkin/zipkin
  restart: always
  environment:
    - STORAGE_TYPE=mem
  ports:
    - 9411:9411
```

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

applicaction.yml o application.properties

```
management.tracing.sampling.probability=1.0
```

Agregar Logger donde se quiera ver su trazabilidad

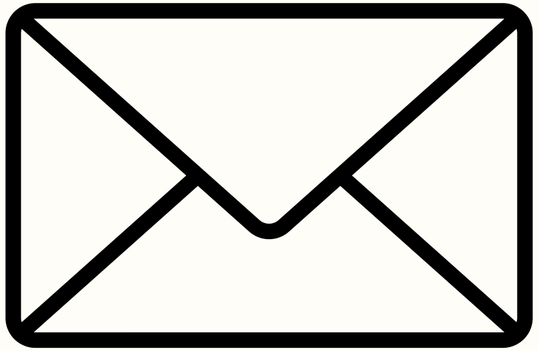
```java
@Component  joseLaptop *
public class SampleGlobalFilter implements GlobalFilter, Ordered {

    private final Logger logger = LoggerFactory.getLogger(SampleGlobalFilter.class);  2 usages

    @Override  joseLaptop *
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        //PRE

        logger.info("Ejecutando global filter antes del request PRE");

        exchange.getRequest().mutate().headers( HttpHeaders header -> header.add( headerName: "token",  headerValue: "PRE-TOKEn"));

        //POST
        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            logger.info("Ejecutando global filter request POST");

            //Programación Funcional
            Optional.ofNullable(exchange.getRequest().getHeaders().getFirst( headerName: "token")).ifPresent( String value -> {
                exchange.getResponse().getHeaders().add( headerName: "token", value);
            });

            exchange.getResponse().getCookies().add("color", ResponseCookie.from( name: "color",  value: "red").build());
            //exchange.getResponse().getHeaders().setContentType(MediaType.TEXT_PLAIN);
        }));
```

Configurar logback-spring.xml

```xml
<configuration>
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} | %5p | %X{traceId} |%X{spanId} | %msg%n</pattern>
        </encoder>
    </appender>
    <root level="info">
        <appender-ref ref="STDOUT"/>
    </root>
</configuration>
```

appender: Define un appender de consola para escribir registros en la consola.

encoder: Formatea los mensajes de registro utilizando el patrón especificado.



%5p: Nivel de registro (e.g., INFO, ERROR).

%msg: El mensaje del log que escribamos en el código.

Datos que faltan en los registros (Programación Reactiva)

```
2025-05-13 11:46:40.749 |  INFO |  | | Ejecutando global filter antes del request PRE
2025-05-13 11:46:41.405 |  INFO |  | | Ejecutando global filter request POST
```

¿Por qué Hooks.enableAutomaticContextPropagation()

Aseguramos que el contexto de seguimiento se propague automáticamente entre hilos y límites asíncronos dentro de los flujos reactivos.

```
@SpringBootApplication  joseLaptop *
@EnableDiscoveryClient
public class GatewayApplication {

    public static void main(String[] args) { SpringApp

    @PostConstruct  new *
    public void init() {
        Hooks.enableAutomaticContextPropagation();
    }
}
```

```
2025-05-13 11:51:17.296 |  INFO | 68237885f39ae0ba35d3ab7481a360e8 |35d3ab7481a360e8 | Ejecutando global filter antes del request PRE
2025-05-13 11:51:17.874 |  INFO | 68237885f39ae0ba35d3ab7481a360e8 |35d3ab7481a360e8 | Ejecutando global filter request POST
```

Zipkin | Find a trace | Dependencies | Search by trace ID | EN

api-gateway: http get

SPAN TABLE

Duration 44.767ms | Services 2 | Total Spans 3 | Trace ID 288ec2019c9103a9da9cf3296885f586

0ms | 14.922ms | 29.845ms | 44.767ms

Focus on selected span | Reset focus

	0ms	14.922ms	29.845ms	44.767ms
2 –	api-gateway: http get			44.767ms
1 –	api-gateway: http get			33.028ms
	order-service: http get /			28.344ms

Service name
api-gateway

Span name
http get

Span ID
06dfdb6fd53c2d1f

Parent ID
none

Annotation

0ms | 14.922ms | 29.845ms | 44.767ms

0ms	Start Time	05/13 12:42:15.042
	Value	Server Start
	Address	172.31.16.1 (api-gateway)
44.767ms	Start Time	05/13 12:42:15.087
	Value	Server Finish
	Address	172.31.16.1 (api-gateway)

Casos reales de uso de la trazabilidad distribuida

Identificación de cuellos de botella de latencia en el procesamiento de pagos

Si una solicitud de procesamiento de pago está tardando más de lo esperado, el seguimiento distribuido le ayuda a ver dónde se está produciendo el retraso.

Si una solicitud falla, OpenTelemetry le ayuda a rastrear el error hasta el servicio exacto e incluso el método específico que causa el fallo.

Comprender las dependencias del sistema

Proporciona un mapa de las dependencias de los servicios, lo que ayuda a los equipos a ver cómo interactúan los servicios entre sí y dónde pueden estar los posibles puntos de fallo

Diagnóstico de errores en flujos de trabajo multiservicio

BIBLIOGRAFÍA

Saxena, A. (2024, noviembre 11). **Distributed Tracing with OpenTelemetry, Spring Cloud Sleuth, Zipkin, and Honeycomb in Spring Boot.** Medium. https://medium.com/@iamayush027/distributed-tracing-with-opentelemetry-spring-cloud-sleuth-zipkin-and-honeycomb-in-spring-boot-37d27ff42b3d

Larsson, M. (2023). Understanding distributed tracing (Cap. 14). **En Microservices with Spring Boot 3 and Spring Cloud** (3.ª ed.). Packt Publishing.

Guzmán Dev, A. [Andrés Guzmán Dev]. (2019). **Introducción a la trazabilidad distribuida con Spring Cloud Sleuth y Zipkin [Video]**. YouTube. https://www.youtube.com/watch?v=9TsXLGpt5Eo

BIBLIOGRAFÍA

Aakash. (202). Understanding distributed tracing (Cap. 14).**Mastering Reactive Tracing in Spring Boot 3: A Comprehensive Guide to Achieving Seamless Observability.** Medium https://medium.com/@aakashkumar2001jha/mastering-reactive-tracing-in-spring-boot-3-a-comprehensive-guide-to-achieving-seamless-29c1229177ca

Muchas

GRACIAS